

Chapter 7

Decomposition based heuristics

7.1 Introduction

Decompositions are methods derived from the “*divide et impera*” principle, dictating to break up a difficult problem into smaller ones, and to solve each of the smaller ones separately, ultimately recomposing the individual solutions to get the overall one. Decompositions have longly been applied to solve optimization problems, and they come in many different flavors, ranging from constraint programming to logical decomposition, from dynamic programming to linear decompositions, among many others.

In this text, we are interested in heuristic approaches. We present three related mathematical decomposition techniques that have been used as seeds for classes of heuristic algorithms, plainly to be included among matheuristics, namely Lagrangian, Dantzig-Wolfe, and Benders decompositions. They are all decompositions that can also be applied to linear problems, and were outlined in Chapter 1 for the case of the GAP, but will be introduced in the next section in some more detail. We will finally give a thorough presentation and a run trace for a Lagrangian heuristic.

7.2 Linear decompositions

Decompositions can be applied to continuous, mixed-integer, and pure integer linear programming problems, but we only introduce here the basic formulae of decomposition in the general case of a mixed-integer problem. The discussion is for a minimization problem, but it is trivial to convert it to a maximization one. Moreover, we are interested in decomposition strategies that lend themselves to seed new heuristic approaches, and this has been reported in the literature mainly for the case of linear problems with a specific structure. We will use this case throughout this chapter.

Decompositions of a mathematical formulation can be trivially obtained when the constraint matrix is block separable. More interesting cases arise when blocks can be identified in the constraint matrix, but they are linked by some constraints (*linking constraints*) or by some variables (*linking variables*). We will concentrate only on some specific case, as the interested reader can anyway refer to the abundant literature on general decomposition in optimization.

In the case of interest, the structure of a linear problem suitable for effective decomposition is therefore that of the following problem LP:

$$z_{LP} = \min \mathbf{c}_1 \mathbf{x}^1 + \mathbf{c}_2 \mathbf{y} + \mathbf{c}_3 \mathbf{x}^2 \quad (7.1)$$

$$\text{s.t. } \mathbf{A} \mathbf{x}^1 + \mathbf{B} \mathbf{y} \geq \mathbf{b} \quad (7.2)$$

$$\mathbf{D} \mathbf{y} + \mathbf{E} \mathbf{x}^2 \geq \mathbf{d} \quad (7.3)$$

$$\mathbf{x}^1, \quad \mathbf{y}, \quad \mathbf{x}^2 \geq \mathbf{0} \quad (7.4)$$

Problem LP could be split into two subproblems, were it not for the linking variables $\mathbf{y}$ (note that in a more general case the problem could split into many subproblems). In fact, the problem becomes block separable in the variables $\mathbf{x}^1$ and $\mathbf{x}^2$ if we fix the variables $\mathbf{y}$ to some values $\bar{\mathbf{y}}$. In this case, problem LP can be solved by solving the two subproblems independently, by minimizing $z_{LP}(\mathbf{x}^1, \mathbf{x}^2, \bar{\mathbf{y}}) = z_{LP1}(\mathbf{x}^1, \bar{\mathbf{y}}) + z_{LP2}(\mathbf{x}^2, \bar{\mathbf{y}})$, where

$$\begin{aligned} z_{LP1} &= \min \mathbf{c}_1 \mathbf{x}^1 & z_{LP2} &= \min \mathbf{c}_3 \mathbf{x}^2 \\ \text{s.t. } \mathbf{A} \mathbf{x}^1 &\geq \mathbf{b} - \mathbf{B} \bar{\mathbf{y}} & \text{s.t. } \mathbf{E} \mathbf{x}^2 &\geq \mathbf{d} - \mathbf{D} \bar{\mathbf{y}} \\ \mathbf{x}^1 &\geq \mathbf{0} & \mathbf{x}^2 &\geq \mathbf{0} \end{aligned}$$

Since the problem becomes separable when $\mathbf{y}$ is fixed, a two-stage method can be used for solving problem LP. If we denote by $z_1(\bar{\mathbf{y}})$ the optimal value of the problem $\min z_{LP1}(\mathbf{x}^1, \bar{\mathbf{y}})$ – henceforth denoted *subproblem 1* – and similarly, by $z_2(\bar{\mathbf{y}})$ the optimal value of the problem $\min z_{LP2}(\mathbf{x}^2, \bar{\mathbf{y}})$ – henceforth denoted *subproblem 2* – then the original problem LP is equivalent to the problem $\min_{\mathbf{y}} (z_1(\mathbf{y}) + z_2(\mathbf{y}))$.

The problem of fixing the $\mathbf{y}$ variables to eventually achieve global optimality is called the *master problem*. The variables of the master problem are therefore the linking (complicating) variables of the original problem and its objective derives from the sum of the optimal values of the subproblems.

A generic primal decomposition method solves problem LP by iteratively solving the master problem. Each iteration fixes the linking variables and proceeds solving the two subproblems independently, in order to evaluate $z_1(\mathbf{y})$ and $z_2(\mathbf{y})$, obtaining information on how to update the linking variables at the next master iteration.

Such a decomposition method is effective when there are few complicating variables, and there are efficient algorithms for solving the subproblems.

This basic decomposition method is called *primal decomposition* because the master problem and the subproblems are defined only on the primal variables. If dual problems are instead used, we deal with dual decompositions, an example of which will be explored in a following section.

We will not further detail this procedure, available in standard optimization textbooks, but we will describe it for the case of interest for matheuristic design, which so far contemplated only reports on the case where, after fixing the linking variables in the master problem, only one subproblem is left to solve, i.e., there are no $\mathbf{x}^2$ variables. However, we include in the presentation the Mixed Integer Linear Programming (MILP) case, where the master problem variables are required to be integer. In this case, the problem to solve, called P, can be represented according to the following structure:

$$z_P = \min \mathbf{c}_1 \mathbf{x} + \mathbf{c}_2 \mathbf{y} \quad (7.5)$$

$$s.t. \mathbf{Ax} + \mathbf{By} \geq \mathbf{b} \quad (7.6)$$

$$\mathbf{Dy} \geq \mathbf{d} \quad (7.7)$$

$$\mathbf{x} \geq \mathbf{0} \quad (7.8)$$

$$\mathbf{y} \geq \mathbf{0} \text{ and integer} \quad (7.9)$$

We further assume, for ease of presentation, that the feasibility region is non-null and bounded, and that constraints (7.7) are *easy* to satisfy while constraints (7.6) are hard, complicating constraints.

In the following subsections we introduce three different ways to decompose problem P either by primal or by dual decomposition: Dantzig-Wolfe, Lagrangian, and Benders decompositions.

7.2.1 Dantzig-Wolfe decomposition

Dantzig-Wolfe decomposition is an iterative procedure that successively approximates the linear relaxation of problem P by decomposing it into a sequence of smaller and/or easier *subproblems*, in accordance with the general working introduced in the previous section. The subproblems dynamically generate the columns of the *master problem* corresponding to the linear programming relaxation of P.

In the case of our example, problem P, we keep the difficult linking constraints (7.6) in the master problem, as introduced in Section 7.2, and demand the rest to the subproblems. Let F be the feasible region induced by constraints (7.7)–(7.9), i.e. $F = \{(\mathbf{x}, \mathbf{y}) : \mathbf{Dy} \geq \mathbf{d}, \mathbf{x} \geq \mathbf{0}, \mathbf{y} \geq \mathbf{0} \text{ and integer}\}$, which we assume finite and non-null, and let $\{(\mathbf{x}^t, \mathbf{y}^t) : t = 1, \dots, T\}$ be the set of the extreme points of F .

Dantzig-Wolfe proceeds by identifying optimal (with respect to the current cost function) extreme points of F , computed as solutions of the *subproblem*, then passing them to the *master problem* in order to check them against the relaxed constraints, i.e., the linking constraints, which are not F -defining. The master problem is formulated by replacing F with the convex combination of its extreme points. Since we have assumed that F is non-null and bounded, we do not have to consider the extreme rays.

After having computed the cost of the best combination of the so far proposed extreme points of F , taking into consideration also the relaxed constraints retained in the master, the subproblem costs are updated, and are computed as reduced costs derived from the values of the dual variables associated to the relaxed constraints. The subproblem is then solved again, to see whether a new, less expensive extreme point can be found.

In the case of problem P, a possible master problem, obtained by relaxing the “difficult” linking constraints (7.6), thus keeping them in the master problem, is as follows:

$$z_{MDW} = \min \sum_{t=1}^T (\mathbf{c}_1 \mathbf{x}^t + \mathbf{c}_2 \mathbf{y}^t) \mu_t \quad (7.10)$$

$$s.t. \sum_{t=1}^T (\mathbf{A} \mathbf{x}^t + \mathbf{B} \mathbf{y}^t) \mu_t \geq \mathbf{b} \quad (7.11)$$

$$\sum_{t=1}^T \mu_t = 1 \quad (7.12)$$

$$\mu_t \geq 0, \quad t = 1, \dots, T \quad (7.13)$$

The corresponding subproblem is:

$$z_{SDW}(\mathbf{u}, \alpha) = \min (\mathbf{c}_1 - \mathbf{u} \mathbf{A}) \mathbf{x} + (\mathbf{c}_2 - \mathbf{u} \mathbf{B}) \mathbf{y} - \alpha \quad (7.14)$$

$$s.t. \mathbf{D} \mathbf{y} \geq \mathbf{d} \quad (7.15)$$

$$\mathbf{x} \geq \mathbf{0} \quad (7.16)$$

$$\mathbf{y} \geq \mathbf{0} \text{ and integer} \quad (7.17)$$

where $\mathbf{u}$ and α are the dual variables corresponding to constraints (7.11) and (7.12) of the master problem, respectively.

If the optimal solution $(\mathbf{x}^*, \mathbf{y}^*)$ of the subproblem has a cost $z_{SDW}(\mathbf{u}, \alpha) < 0$, we can add into the master problem the column of cost $(\mathbf{c}_1 \mathbf{x}^* + \mathbf{c}_2 \mathbf{y}^*)$, having $(\mathbf{A} \mathbf{x}^* + \mathbf{B} \mathbf{y}^*)$ corresponding to constraint (7.11), and a 1 coming from constraint (7.12), otherwise we have reached the optimal solution of MDW.

At each iteration of the procedure, a valid lower bound to the optimal solution value of the original problem is given by $z_{MDW} + z_{SDW}$. This lower bound is not monotonically nondecreasing, therefore we need to maintain the best value obtained along the iterations.

7.2.2 Lagrangian Relaxation and Decomposition

A second decomposition is obtained by a *Lagrangian relaxation*, which permits to compute a lower bound to problem P by removing some difficult constraints and by dualizing them into the objective function by means of *Lagrangian penalties*.

In the example of problem P, we can again relax the difficult constraints (7.6), this time using the non-negative Lagrangian penalty vector λ , thus obtaining the following formulation LR:

$$z_{LR}(\lambda) = \min \mathbf{c}_1 \mathbf{x} + \mathbf{c}_2 \mathbf{y} + \lambda (\mathbf{b} - \mathbf{A}\mathbf{x} - \mathbf{B}\mathbf{y}) \quad (7.18)$$

$$s.t. \mathbf{D}\mathbf{y} \geq \mathbf{d} \quad (7.7)$$

$$\mathbf{x} \geq \mathbf{0} \quad (7.8)$$

$$\mathbf{y} \geq \mathbf{0} \text{ and integer} \quad (7.9)$$

The value $z_{LR}(\lambda)$ is a valid lower bound to the optimal value of P, i.e., $z_{LR}(\lambda) \leq z_P$, for every $\lambda \geq \mathbf{0}$.

To identify the penalty vector λ that maximizes the lower bound $z_{LR}(\lambda)$ we solve a dualized master problem, in this case the so-called *Lagrangian Dual*, that can be formulated as follows:

$$z_{LR} = \max \{z_{LR}(\lambda) : \lambda \geq \mathbf{0}\} \quad (7.19)$$

which fixes a penalty vector and then asks to solve the internal *subproblem* LR. This can be rewritten as follows:

$$z_{LR}(\lambda) = \min (\mathbf{c}_1 - \lambda \mathbf{A})\mathbf{x} + (\mathbf{c}_2 - \lambda \mathbf{B})\mathbf{y} + \lambda \mathbf{b} \quad (7.20)$$

$$s.t. \mathbf{D}\mathbf{y} \geq \mathbf{d} \quad (7.21)$$

$$\mathbf{x} \geq \mathbf{0} \quad (7.22)$$

$$\mathbf{y} \geq \mathbf{0} \text{ and integer} \quad (7.23)$$

If the subproblem is solved to integrality, it is possible that the lower bound provided by z_{LR} is tighter than the linear relaxation of problem P.

Notice that it is possible to add to subproblem LR additional constraints that are redundant in the original formulation, but that can help convergence. Moreover, it is sometimes possible to obtain feasible dual solutions directly from the Lagrangian penalties. Approaches based on this property have been used, for example, to generate reduced problems that consider only the variables of k -least reduced cost. These approaches can make it possible to substantially reduce the number of variables to consider when solving the problem heuristically, or in some cases also to optimality. The k -least reduced cost variables can also be generated in a column-generation fashion, combining Lagrangian relaxation and column generation.

Subproblem relations

It is intriguing to note that subproblem LR is identical to subproblem SDW obtained in the Dantzig-Wolfe decomposition, if we replace $\mathbf{u}$ and α with λ and $-\lambda \mathbf{b}$, respectively. Such identities among subproblems obtained by different relaxations will be outlined also in the following sections.

7.2.3 Benders decomposition

Benders decomposition computes a lower bound to the optimal cost of the original problem again by solving a *master problem* that fixes some linking variables. Then, to improve the lower bound, it solves a *subproblem*, which adds new constraints to the master. The approach is quite coherent with the description proposed in Section 7.2, except that it is defined on the dual.

The original problem (7.5)–(7.9) can be rewritten splitting it into a master problem z_B :

$$z_B = \min \mathbf{c}_2 \mathbf{y} + z_{SB}(\mathbf{y}) \quad (7.24)$$

$$s.t. \mathbf{D} \mathbf{y} \geq \mathbf{d} \quad (7.25)$$

$$\mathbf{y} \geq \mathbf{0} \text{ and integer} \quad (7.26)$$

and the subproblem $z_{SB}(\mathbf{y})$:

$$z_{SB}(\mathbf{y}) = \min \mathbf{c}_1 \mathbf{x} \quad (7.27)$$

$$s.t. \mathbf{A} \mathbf{x} \geq \mathbf{b} - \mathbf{B} \mathbf{y} \quad (7.28)$$

$$\mathbf{x} \geq \mathbf{0} \quad (7.29)$$

The dual of the subproblem $z_{SB}(\mathbf{y})$ is

$$z_{SBD}(\mathbf{y}) = \max \mathbf{w}(\mathbf{b} - \mathbf{B} \mathbf{y}) \quad (7.30)$$

$$s.t. \mathbf{w} \mathbf{A} \leq \mathbf{c}_1 \quad (7.31)$$

$$\mathbf{w} \geq \mathbf{0} \quad (7.32)$$

Upon denoting $W = \{\mathbf{w} : \mathbf{w} \mathbf{A} \leq \mathbf{c}_1, \mathbf{w} \geq \mathbf{0}\}$ and $Y = \{\mathbf{y} : \mathbf{D} \mathbf{y} \geq \mathbf{d}, \mathbf{y} \geq \mathbf{0} \text{ and integer}\}$, we can rewrite problem B as follows:

$$z_B = \min_{\mathbf{y} \in Y} \max_{\mathbf{w} \in W} (\mathbf{c}_2 - \mathbf{w} \mathbf{B}) \mathbf{y} + \mathbf{w} \mathbf{b}. \quad (7.33)$$

Let $\{\mathbf{w}^t, t = 1, \dots, T\}$ be the set of the extreme points of W . Since we have assumed the feasible region finite and non-null, we can express z_B as a function of the extreme points as follows:

$$z_B = \min_{\mathbf{y} \in Y} \max_{t=1, \dots, T} (\mathbf{c}_2 - \mathbf{w}^t \mathbf{B}) \mathbf{y} + \mathbf{w}^t \mathbf{b} \quad (7.34)$$

which is equivalent to the following formulation:

$$z_{MB} = \min z \quad (7.35)$$

$$s.t. z \geq (\mathbf{c}_2 - \mathbf{w}^t \mathbf{B}) \mathbf{y} + \mathbf{w}^t \mathbf{b}, \quad t = 1, \dots, T \quad (7.36)$$

$$\mathbf{y} \in Y \quad (7.37)$$

Problem MB is the *Benders' master problem* and constraints (7.36) are the so-called *Benders' cuts*. The number of Benders' cuts T is usually huge, therefore the master problem is initially solved considering only a small number T' of Benders' cuts, i.e., $T' \ll T$. In order to ascertain whether the solution is already optimal or an additional cut should be added to the master, we need to solve a *subproblem* $z_{SB}(\mathbf{y})$ defined in (7.27)–(7.29).

Subproblem relations

We have already remarked that Dantzig-Wolfe and Lagrangian decomposition give rise to almost identical subproblems. It is noteworthy to show that also for Benders decomposition we can have a subproblem similar to the two already seen ones. In fact, if we rewrite problem (7.33) as follows

$$z_B = \max_{\mathbf{w} \in W} \min_{\mathbf{y} \in Y} (\mathbf{c}_2 - \mathbf{wB})\mathbf{y} + \mathbf{wb} \quad (7.38)$$

it is easy to note that the inner optimization $\min_{\mathbf{y} \in Y} (\mathbf{c}_2 - \mathbf{wB})\mathbf{y} + \mathbf{wb}$ is almost equivalent to the Dantzig-Wolfe and Lagrangian subproblems. The vector $\mathbf{w}$ can replace the dual variable vector $\mathbf{u}$ in the Dantzig-Wolfe decomposition or the penalty vector $\boldsymbol{\lambda}$ in the Lagrangian relaxation. Moreover, since $\mathbf{w}$ is the optimal dual solution of the subproblem $z_{SB}(\mathbf{y})$ defined in (7.30)–(7.32), due to the complementary-slackness, we have in the Benders as in the other decompositions that $(\mathbf{c}_1 - \mathbf{wA})\mathbf{x} = 0$.

7.3 Matheuristics derived from decompositions

All three decompositions, besides possibly other ones such as surrogate approaches, can be used as foundations upon which to design general heuristic cores and which could be easily adapted to a combinatorial optimization problem of interest. The literature detailedly describes how this can be done in each case. Here we present only the case for Lagrangian relaxation and how it can be specified for the GAP.

7.3.1 *Lagrangian metaheuristic*

The literature is rich with heuristics based on the decomposition structure outlined in Section 7.2. Among them, Lagrangian relaxation has a particularly good record of successful applications.

Most Lagrangian heuristics share a common general structure, that can be considered a metaheuristic of its own. The structure is presented in Algorithm 35.

The pseudocode of algorithm 35 must be specified for any specific application. It has been observed that it is presented at the abstraction level where metaheuristics are usually introduced. In fact, this pseudocode already shows the essential ingredients of metaheuristics, which is defined as “an iterative master process that

Algorithm 35: Core Lagrangian heuristic

```

1 function LagrHeuristic();
   Input : Control parameters
   Output: A feasible solution  $\mathbf{x}^*$  of value  $z^*$ 
2 Initialize the penalty vector  $\boldsymbol{\lambda}$ ;
3 Identify an “easy” subproblem  $\text{LR}(\boldsymbol{\lambda})$ ;
4 repeat
5     solve subproblem  $\text{LR}(\boldsymbol{\lambda})$  obtaining the possibly infeasible solution  $\mathbf{x}$ ;
6     check for unsatisfied constraints;
7     update penalties  $\boldsymbol{\lambda}$ ;
8     construct problem solution  $\mathbf{x}^h$  using  $\mathbf{x}$  and  $\boldsymbol{\lambda}$ ;
9     if  $z(\mathbf{x}^h) < z^*$  then  $\mathbf{x}^* = \mathbf{x}^h$ ;           //  $z(\mathbf{x}^h), z^*$  are the costs of  $\mathbf{x}^h, \mathbf{x}^*$ 
10 until end_condition;

```

guides and modifies the operations of a subordinate heuristic”, where the subordinate heuristic is at step 8.

Steps 3 and 5 are problem-dependent, much alike neighborhoods definition or crossover implementation in other contexts. Step 6 is trivial, while Step 7 can be implemented by means of any state of the art techniques, usually subgradient optimization or bundle methods. Interestingly, some of these techniques have been proved to converge not only to the optimal $\boldsymbol{\lambda}$, but also to the optimal $\mathbf{x}$ of the linear relaxation, thereby possibly providing a particularly “intelligent” starting point for Step 8.

7.3.2 Lagrangian matheuristics

Algorithm 35 proposes the Lagrangian heuristic as an add-on to a core procedure that computes the Lagrangian bound. This is, in fact, the way most Lagrangian heuristics have usually been proposed, and the literature describing these approaches is rich and decades-long.

The core of the heuristics is therefore the algorithm that computes the Lagrangian bound. The most used algorithms for this are the following:

- *Subgradient optimization*, is a simple algorithm for minimizing a possibly non-differentiable convex function. The method is similar to an ordinary gradient method for differentiable functions but departs from it in several important details. For example, it can use precomputed step lengths, instead of an exact or approximate line search and, more importantly, it does not use a direction which is an actual gradient, as obtained from the differentiation of the objective function, but the direction is obtained from the violations of the relaxed constraints, violations which can define “subgradients”. Moreover, the subgradient algorithm is not a descent method, the function value can repeatedly increase and decrease along its iterations, as it usually does.

- *Multiplier adjustment*, interestingly introduced with an application to the GAP, increases one multiplier at a time. The method begins by initializing each multiplier λ_j to its second largest c_{ij} , so that $c_{ij} - \lambda_j > 0$ for at most one $i \in I$ and such that there is an optimal Lagrangian solution for this λ that satisfies $\sum_{i \in I} x_{ij} \leq 1$ for $j \in J$. If $\sum_{i \in I} x_{ij} = 1$ for all $j \in J$, then this solution is feasible and optimal. Otherwise, it is usually possible to select a client j^* for which $\sum_{i \in I} x_{ij^*} = 0$ and decrease λ_{j^*} to a value for which the new Lagrangian solution satisfies $\sum_{i \in I} x_{ij^*} = 1$ and $\sum_{i \in I} x_{ij} \leq 1$ for all other j . If the required conditions are satisfied, the multiplier λ_{j^*} is decreased and the procedure is iterated.
- *Volume algorithm*, is a refinement of the subgradient approach but applied to a Dantzig-Wolfe reformulation of the problem. The algorithm attempts to find an approximate solution of the master problem by computing at each iteration a *stability center*, a *step*, and a subgradient-based *direction*. The stability center represents a point that has provided significant improvement with respect to the optimization process. In turn, the step represents how far one may move in the identified direction so that a new trial point is obtained. The stability center is updated when specific conditions are met. It is worthwhile in our context to notice that, since Lagrangian penalties are related to dual variables of the problem to solve, the algorithm maintains a set of dual variables of the dualized constraints, i.e., a set of primal variables. In this case, it has been observed that when “no further improvement is found, the values (of the primal variables) should approximate an optimal solution”.
- *Bundle methods* are methods where subgradient directions from past iterations are accumulated in a bundle, and a trial direction is usually obtained by solving a quadratic programming model based on the information contained in the bundle. A line search can then be performed along the tentative direction, generating a non-null step if the function value is improved by some positive value, a null step otherwise.

Based on these cores, the literature reporting applications of algorithms, which are structured like algorithm 35, is rich, and includes contributions that expand the area of optimization in general, not being limited to the focus on heuristics. In this section, we limit to reporting about some Lagrangian heuristics proposed specifically for the GAP.

A heuristic based on the subgradient optimization of the Lagrangian relaxation of the GAP was proposed in two publications, which work on different relaxations: a Lagrangian decomposition separating two subproblems and keeping both constraint sets in the first case and a relaxation of the capacity constraints in the second one, but the heuristic is very similar in both cases. The heuristic is applied at each iteration of the subgradient algorithm and tries to construct a feasible client assignment by a greedy approach based on the ordering obtained using increasing costs $f_{ij} = c_{ij} - q_{ij}\lambda_i$, where λ_i is the incumbent penalty associated with the i -th relaxed capacity constraint. If all clients can be assigned, then the solution is feasible, otherwise, every unassigned client j is tentatively assigned to the server i to which corresponds the maximal value of f_{ij} . Next, an interchange heuristic tries to transform this assignment into a feasible one. This is done by a simple fixing heuristic,

which computes a (non-negative) feasibility measure defined to be the sum of the resource shortages of the servers, and attempting to identify a better solution either by interchanging two client / server assignments or by shifting a client to another server. If a feasible assignment is obtained, the same interchange heuristic is used as a local optimization procedure.

The possibility of recovering the feasibility of the Lagrangian bound solution has been studied by including a data perturbation module, denoted as *problem space search* in the original work, where the authors relax the capacity constraints, run a subgradient and try to restore feasibility of the bound solution by means of a greedy constructive heuristic which assigns the clients one by one to servers with enough capacity, making sure that the overall assignment is always feasible. Two different orders of assignment of the clients and the criterion used to assign are presented, but the main contribution is in the data perturbation component. In fact, since it is possible that the feasibility restoration heuristics stop without finding a feasible solution, the penalized costs are temporarily perturbed and the constructive heuristics are run on the modified costs, in the hope that the new cost configuration permits the construction of solutions that are feasible for the original instance. The perturbation is applied in each iteration of subgradient optimization for a given number of times.

Finally, it is worth mentioning here a work not directly applied to the GAP but to the *many-to-many assignment problem*, a problem with a formulation very similar to the GAP, except that the assignment constraints are replaced by knapsack constraints, i.e., clients have capacities and multiple servers can partially serve one same client. The work proposes a modified Lagrangian bound and a greedy heuristic to get a feasible Lagrangian-based solution, a heuristic that relies on the observation that in this problem all constraints are actually inequalities.

7.4 A Lagrangian matheuristic for the GAP

Algorithm 35 is a generic framework for combinatorial optimization problems and must be detailed for the specific problem to solve, in our case the GAP. Steps 3 and 5 are, in fact, non-trivially problem-dependent, while steps 6, 7 and 8 also depend on the problem, but are expected in whichever algorithm is used to solve it. Step 6 is trivial, while step 7 can be implemented by means of any state of the art techniques. Step 8 is where the actual primal solution is found. The end condition is usually set on the number of iterations of the *repeat-until* loop, on a maximum CPU time, on a maximum number of non-improving solutions, or on the minimal value of some internal coefficient, such as the step size coefficient α used in step 7.

In our case, these problem-dependent steps were implemented as follows.

LagrHeuristic: step 3

Section 1.2.2 introduced the two possible Lagrangian relaxations for the GAP: relaxing assignment constraints (1.2) or capacity constraints (1.3). The second possibility leaves a trivial subproblem, which can be solved by inspection but which cannot provide bounds better than the linear relaxation of the original GAP. Relaxing the assignment constraints on the contrary leaves a combinatorial subproblem, therefore the Lagrangian solution can be better than the linear relaxation.

In this subsection we show a detailed trace of the execution of a Lagrangian matheuristic implementing the second possibility, i.e., solving a subproblem derived from the relaxation of the assignment constraints. We copy here the formulation, already presented in Section 1.2.2, for ease of reference.

$$(LGAP) \quad z_{LGAP} = \min \sum_{i \in I} \sum_{j \in J} (c_{ij} + \lambda_j) x_{ij} - \sum_{j \in J} \lambda_j \quad (1.5)$$

$$s.t. \quad \sum_{j \in J} q_{ij} x_{ij} \leq Q_i, \quad i \in I \quad (1.6)$$

$$x_{ij} \in \{0, 1\}, \quad i \in I, j \in J \quad (1.7)$$

$$\lambda_j \text{ unrestricted}, \quad j \in J \quad (1.8)$$

LagrHeuristic: step 5

The subproblem decomposes into $|I|$ knapsack problems, with objective function

$$z_K^i = \sum_{j \in J} (c_{ij} + \lambda_j) x_{ij}$$

where the Lagrangian penalties λ_j are associated with the assignment constraints (1.2). The subproblem solution is simply given by setting $x_{ij} = 1$ for every element j which is chosen to enter the i -th knapsack.

LagrHeuristic: step 6

The solution can have clients assigned to multiple or to no server, the case can be ascertained by direct inspection.

LagrHeuristic: step 7

The quality of the bound obtained by the Lagrangian relaxations heavily depends on the specific penalty vector λ used. The problem of finding a penalty vector that maximizes the Lagrangian bound is the Lagrangian Dual problem. In our case, we

used a standard subgradient procedure for determining the penalties. The procedure computes, at each iteration k , a new approximation λ^{k+1} of the Lagrangian multipliers in such a way that, for $k \rightarrow \infty$, λ^k is an optimal or a near-optimal solution of the corresponding Lagrangian Dual.

The subgradient algorithm implements a local search in the space of the Lagrangian penalties, where the direction of the move at each iteration k is given by a subgradient g^k , and the length of the move, i.e., the step size, is dictated by a scalar α^k as follows:

$$\sigma^k = \alpha^k \frac{\bar{z} - z_{LR}(\lambda^k)}{\|g^k\|^2} \quad (7.39)$$

where $\bar{z}$ is an overestimate of the optimal Lagrangian Dual solution value $z_{LR}(\lambda^*)$, possibly an upper bound to the optimal problem cost, and the components of the subgradient are computed as $g_j^k = \sum_{i \in I} x_{ij} - 1$. In the literature, several different step size rules have been proposed, each one with different convergence results. We adapted in formula (7.39) the seminal rule proposed by Polyak (1969). The step size coefficient α^k is updated following a *diminishing step size* policy.

Given an incumbent solution x^k , the corresponding bound is $z_{LR}(\lambda^k) = z_{LGAPA}(\lambda^k)$. Having these, the Lagrangian multipliers can be updated by setting

$$\lambda^{k+1} = \lambda^k + \sigma^k g^k. \quad (7.40)$$

where σ^k is the length of the step along the search direction given by the subgradient.

LagrHeuristic: step 8

The solutions provided by the inner loop of Algorithm 35 are often infeasible for the reasons mentioned in step 6. The problem of recovering feasibility from a partial solution could be framed as a GAP itself, but this would complicate the solution approach. Moreover, it is known that searching for a GAP feasible solution, or fixing a partially feasible one, is also an NP-hard problem as it is NP-complete the decision problem of determining whether a GAP instance has a feasible solution or not (see Chapter 1). We, therefore, used the approach of Section 3.7.1 as detailed in the following section 7.4.1, which can handle both causes of infeasibility.

In case a feasible solution is found, the Lagrangian heuristic passes it on to a simple local search. The local optimization can be applied to every feasible solution found, and it reassigned, when feasible and convenient, the clients in decreasing regret order to the servers.

After including all elements listed in the previous section into algorithm 35, we obtain the following algorithm 36, which describes a Lagrangian matheuristic specialized for the GAP.

Algorithm 36: Lagrangian matheuristic for the GAP

```

1 function LagrHeuristicGAP( $\alpha$ );
   Input :  $\alpha$  (see formula 7.40)
   Output: A feasible solution  $\mathbf{x}^*$  of value  $z^*$ 
2 set  $z_{lb} = z_{lbBest} = -\infty$ ;           // iteration and best lower bounds
3 set  $z_{curr} = z_{ub} = \infty$ ;           // iteration and best upper bounds
4 Initialize the penalty vector  $\boldsymbol{\lambda}$ ;           // f.e. to a zero vector
5 Use problem LGAPA (see 1.2.2) as subproblem LR( $\boldsymbol{\lambda}$ );
6 repeat
7   solve subproblem LR( $\boldsymbol{\lambda}$ ) by solving the  $m$  knapsack problems it is composed by,
   obtain iteration bound  $z_{lb}$  and solution  $\mathbf{x}_{lb}$ ;
8   if  $z_{lb} < z_{lbBest}$  then  $z_{lbBest} = z_{lb}$ ;           // lower bound update
9   check for unsatisfied assignment constraints;
10  construct problem solution  $\mathbf{x}_h$  using  $\mathbf{x}$  and  $\boldsymbol{\lambda}$  by means of algorithm 20;
11   $z_{curr} = z(\mathbf{x}_h)$ ;           //  $z(\mathbf{x}_h)$  is the cost of  $\mathbf{x}_h$ 
12  if  $z_{curr} < z_{ub}$  then           // upper bound update
13     $\mathbf{x}^* = \mathbf{x}_h$ ;  $z_{ub} = z_{curr}$ ;
14  end
15  update penalties  $\boldsymbol{\lambda}$  by formula (7.40);
16 until end_condition;

```

7.4.1 The fixing heuristic

The fixing heuristic we applied within *LagrHeuristicGAP*() is algorithm 20, presented in Section 3.7.1, which is designed to aggressively try to produce feasible solutions even starting from heavily infeasible ones, and to be computationally reasonably light. This comes at a cost, in this case of being unable to recover feasibility in several cases where a more sophisticated algorithm would be able to. However, the primary concern of algorithm *LagrHeuristicGAP*() is to improve the bound, and this is expected to be achieved by a high number of internal iterations, therefore, it is sufficient to get feasible solutions out of only a percentage of them to ensure the effectiveness of the approach.

This fixing heuristic could be used both to repair infeasibilities deriving from the relaxation of constraints (1.2) and (1.3). We detail it in algorithm 20 for the general case where both assignment and capacity constraints could be unsatisfied in the current Lagrangian solution, even though in the context of this section only assignments can make the solution infeasible.

7.4.2 LagrHeuristic: computational trace

We implemented algorithm 36 coupled with algorithm 20 and ran it on instance *example8x3* of section 1.1. The instance is a simple one, and algorithm 36 was able to solve it to proven optimality, though this is seldom the case with hard instances. The reported run had parameter α (see eq. 7.39) initialized to 2.5.

The first iteration does not assign any client to any server, since any assignment would incur a positive cost. Therefore at iteration 1 we had $z_{lb} = z_{lbBest} = 0.00$ and the subgradient vector is $\mathbf{g}^1 = (1, 1, 1, 1, 1, 1, 1, 1)$, as every assignment constraint should sum to 1 but sums to 0. The subproblem solution

$$\mathbf{x}_{lb} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (7.41)$$

(null assignment) is passed on to algorithm 20, which is very aggressive in its tentative to recover feasibility, and manages to construct the solution reported in (7.42) of cost 336, therefore $z_{ub} = z_{curr} = 336$.

$$\mathbf{x}_h = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \quad (7.42)$$

Since $z_{LR}(\boldsymbol{\lambda}^1) = 0$ and there is a big gap between upper and lower bound – say, big means that the upper bound is more than 20% over the lower bound – it is computationally convenient to modify formula (7.39) and use at the numerator of the fraction a value not much greater than the lower bound, for example, $1.2 * z_{lb} + 1$, instead of the difference among the bounds. We therefore obtain $\bar{z} = 1$, and consequently $\sigma^1 = 2.5 * 1/8 = 0.313$. The final step of the iteration updates all multipliers starting from 0 with the product of σ^1 and the respective subgradient, which is 1 for all of them. Therefore we get $\boldsymbol{\lambda}^1 = (0.313, 0.313, 0.313, 0.313, 0.313, 0.313, 0.313, 0.313)$.

Iteration 2 is similar to iteration 1, except for the higher initial multiplier values. The Lagrangian bound is given by the sum of the penalties, therefore $z_{lb} = z_{lbBest} = 8 * 0.313 = 2.5$. The bound solution is still the null solution, therefore $z_{ub} = 336$ and $\alpha = 2.5$, unchanged. The subgradient vector is again $\mathbf{g}^2 = (1, 1, 1, 1, 1, 1, 1, 1)$, leading to $\boldsymbol{\lambda}^2 = (0.625, 0.625, 0.625, 0.625, 0.625, 0.625, 0.625, 0.625)$ with $\sigma^2 = 0.313$.

Penalties keeps uniformly summing up until iteration 10, which is the first where the sum of the penalties exceeds the cost of an assignment (for client 1 to server 1). This leads the first row of the cost matrix, containing costs minus sum of penalties for server 1 (see equation 1.5), to become $\mathbf{c}^{[1]} = [-0.68, 0.32, 1.32, 2.32, 3.32, 4.32, 5.32, 6.32]$, so that the subproblem solution results to be

$$\mathbf{x}_{lb} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (7.43)$$

of cost $z_{lb} = z_{lbBest} = 84.751$ and thus subgradient vector $\mathbf{g}^{10} = (0, 1, 1, 1, 1, 1, 1, 1)$.

The bound solution is infeasible, and it is fixed obtaining again upper bound $z_{ub} = 336$. This is still too different from the lower bound to be used as $\bar{z}$, and the penal-

ties become $\lambda^{10} = (10.679, 16.732, 16.732, 16.732, 16.732, 16.732, 16.732, 16.732)$ with $\sigma^{10} = 6.054$.

We have to wait until iteration 44 in order to have an improvement of the upper bound. At this iteration the subproblem yields a cost $z_{lb} = 255.808$, even though previously we got a $z_{lbBest} = 275.779$, which testifies that the bound is not subject to a monotonic increase. The bound solution is

$$\mathbf{x}_{lb} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \quad (7.44)$$

which is converted by algorithm 20 into the feasible solution

$$\mathbf{x}_h = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \quad (7.45)$$

of cost $z_{curr} = z_{ub} = 325$. At this iteration we have $\alpha = 2.25$ and $\sigma^{44} = 8.222$. Given solution $\mathbf{x}_{lb}$, the subgradient vector at this iteration is $\mathbf{g}^{44} = (1, 1, 1, -1, 1, -2, -2, 1)$, computed on solution (7.44).

Finally, at iteration 830, the subproblem produces the solution

$$\mathbf{x}_{lb} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \quad (7.46)$$

which is still infeasible, but it has a cost $z_{lb} = z_{lbBest} = 324.327$. This guarantees that an optimal solution must have a cost of at least 325, given the integrality constraints and the fact that all costs are integer. Since we already found at iteration 44 a feasible solution of cost 325, we have the proof that the best found solution is indeed an optimal one, and we can stop the search.

To complete the presentation of this run, Figure 7.1 shows the trace of the evolution of the upper and lower bounds. It is apparent how the Lagrangian lower bound shows a non-monotonic increase along the iterations, with decreasing variance, hence the need to store a (monotonically increasing) z_{lbBest} . The upper bound z_{curr} is not produced at each iteration, dots show it in correspondence to the iterations where it could be found. In this run, the subproblem was never able to produce a solution feasible for the whole problem (which would have been optimal), and the fixing heuristic, though aggressive, is often unable to start from the infeasible Lagrangian solution and bring it to global feasibility.

Moreover, the z_{ub} trace shows the evolution of the cost of the best-so-far solution, which is monotonically nonincreasing. At the last iteration, the value of the costs of the Lagrangian solution and of the best-so-far solution differ for less than a unit, indicating that the best-so-far is also an optimal solution.

Further insight can be obtained by visualizing the traces of relevant parameters along the run. Figure 7.2 shows the evolution of the values taken by each Lagrangian

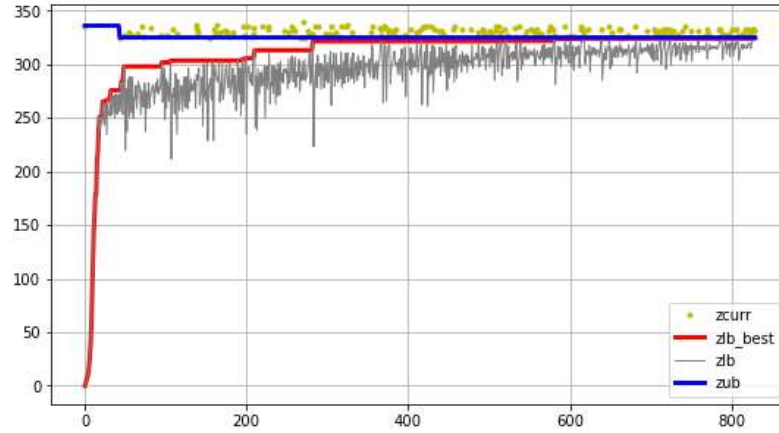


Fig. 7.1 Bound traces

multiplier, they are 8 in the case of this instance. While the evolution of every single multiplier cannot be discerned, it is interesting to notice that all of them follow similar trajectories, where at the beginning the penalties are quickly carried to values that permit to produce tight lower bounds and, afterward, they are slowly fine-tuned each one with respect to all others in order to get the tightest possible bound.

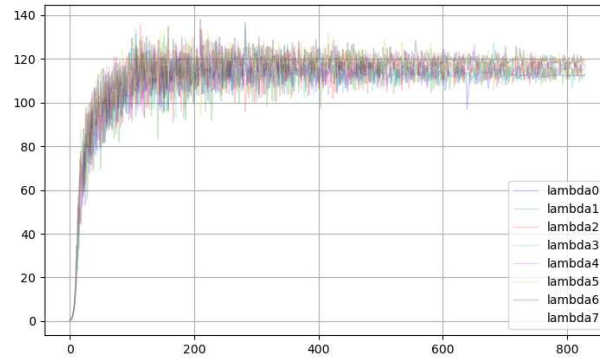


Fig. 7.2 Penalty traces

Finally, Figure 7.3 shows the evolution of the step length σ^k , which determines the magnitude of the multiplier update internally to the subgradient optimization

algorithm. This result is obtained by formula (7.39), using the value of the α calling argument and of the subgradient vector of the current iteration.

Again, we can notice a very fast increase to useful values, followed by a slow, nonmonotonic decrease that permits the cautious update of the multipliers and, ultimately, the identification of the optimality bound.

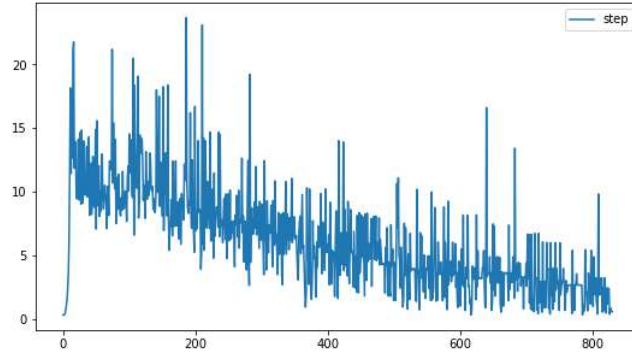


Fig. 7.3 Subgradient step length trace

7.5 Related literature

A good introduction to the topic of decompositions in mathematical programming is in Bazaraa et al (1990). Reviews of literature presenting decomposition-based matheuristics can be found in Boschetti and Maniezzo (2009); Boschetti et al (2009), and Raidl (2015).

Dantzig-Wolfe decomposition was originally introduced in Dantzig and Wolfe (1960), Benders decomposition in Benders (1962).

An excellent introduction to the whole topic of Lagrangian relaxation, and of related heuristics, can be found in Beasley (1993). Approaches obtaining dual values from Lagrangian penalties are reported in Boschetti et al (2008); Boschetti and Maniezzo (2015); Boschetti et al (2020); Mingozzi et al (1999). Results on the convergence on optimal primal values of the linear relaxation were published in Barahona and Anbil (2000) and Sherali and Choi (1996).

Subgradient optimization is presented by Shor (1985), multiplier adjustment by Fisher et al (1986), the volume algorithm by Barahona and Anbil (2000), and bundle methods by Hiriart-Urruty and Lemarechal (1993). The reported subgradient optimization of the Lagrangian relaxation of the GAP was proposed in Haddadi (1999) and in Haddadi and Ouzia (2001) and the fixing of the Lagrangian bound in Jeet and Kutanoglu (2007). The fixing used in the example was originally presented in

Maniezzo et al (2019). The work on the many-to-many assignment problem is by Litvinchev et al (2010).

Subgradient step size rules have been reviewed in Boschetti et al (2011) and Held et al (1974).

References

- Barahona F, Anbil R (2000) The volume algorithm: producing primal solutions with a subgradient method. *Mathematical Programming* 87:385–399
- Bazaraa MS, Jarvis J, Sherali HD (1990) *Linear Programming and Network Flows*. John Wiley & Sons
- Beasley JE (1993) Lagrangean relaxation. In: *Modern heuristic techniques for combinatorial problems*, Reeves, C.R., Blackwell Scientific Publ, pp 243–303
- Benders JF (1962) Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik* 4:280–322
- Boschetti M, Maniezzo V, Roffilli M (2009) Decomposition techniques as metaheuristic frameworks. In: Maniezzo V, Stützle T, Voß S (eds) *Matheuristics*. *Annals of Information Systems*, vol 10, Springer, Boston, MA
- Boschetti MA, Maniezzo V (2009) Benders decomposition, Lagrangian relaxation and metaheuristic design. *Journal of Heuristics* 15(3):283–312
- Boschetti MA, Maniezzo V (2015) A set covering based matheuristic for a real-world city logistics problem. *International Transactions in Operational Research* 22(1):169–195
- Boschetti MA, Mingozzi A, Ricciardelli S (2008) A dual ascent procedure for the set partitioning problem. *Discrete Optimization* 5(4):735–747
- Boschetti MA, Maniezzo V, Roffilli M (2011) Fully distributed Lagrangian solution for a peer-to-peer overlay network design problem. *INFORMS Journal on Computing* 23(1):90–104
- Boschetti MA, Golfarelli M, Graziani S (2020) An exact method for shrinking pivot tables. *Omega* 93:10–44
- Dantzig GB, Wolfe P (1960) Decomposition principle for linear programs. *Operations Research* 8:101–111
- Fisher ML, Jaikumar R, Van Wassenhove LN (1986) A multiplier adjustment method for the generalized assignment problem. *Management Science* 32(9):1095–1103
- Haddadi S (1999) Lagrangian decomposition based heuristic for the generalized assignment problem. *Information Systems and Operational Research* 37:392–402
- Haddadi S, Ouzia H (2001) An effective Lagrangian heuristic for the generalized assignment problem. *INFOR Information Systems and Operational Research* 39:351–356
- Held M, Wolfe P, Crowder HP (1974) Validation of subgradient optimization. *Mathematical Programming* 6(1):162–88

- Hiriart-Urruty JB, Lemarechal C (1993) Convex analysis and minimization algorithms II: Advanced theory and bundle methods. *A Series of Comprehensive Studies in Mathematics* 306
- Jeet V, Kutanoglu E (2007) Lagrangian relaxation guided problem space search heuristics for generalized assignment problems. *European Journal of Operational Research* 182(3):1039–1056
- Litvinchev I, Mata M, Rangel S, Saucedo J (2010) Lagrangian heuristic for a class of the generalized assignment problems. *Computers & Mathematics with Applications* 60(4):1115–1123
- Maniezzo V, Boschetti MA, Carbonaro A, Marzolla M, Strappaveccia F (2019) Client-side computational optimization. *ACM Transactions on Mathematical Software (TOMS)* 45(2):1–16
- Mingozzi A, Boschetti MA, Ricciardelli S, Bianco LA (1999) Set partitioning approach to the crew scheduling problem. *Operations Research* 47:873–888
- Polyak BT (1969) Minimization of unsmooth functionals. *USSR Computational Mathematics and Mathematical Physics* 9:14–29
- Raidl G (2015) Decomposition based hybrid metaheuristics. *European Journal of Operational Research* 244:66–76
- Sherali HD, Choi G (1996) Recovery of primal solutions when using subgradient optimization methods to solve Lagrangian duals of linear programs. *Operations Research Letters* 19:105–113
- Shor NZ (1985) *Minimization Methods for Non-differentiable Functions*. Springer